

Unión de Conjuntos Disjuntos (DSU)

[\[Versión Original en Inglés\]](#)

La estructura de datos **Unión de Conjuntos Disjuntos** (**Disjoint Set Union**, **DSU** o **Union-Find**) es una estructura de datos que almacena una colección de conjuntos disjuntos (que no se superponen). Equivale a mantener una partición de un conjunto de elementos en subconjuntos separados.

1. Operaciones Fundamentales

La DSU está diseñada para soportar eficientemente tres operaciones clave:

a. **make_set(v)** (Crear Conjunto)

Crea un nuevo conjunto que contiene solo el elemento v . Inicialmente, cada elemento comienza como un conjunto individual.

- **Representación:** El elemento v es su propio padre (o representante).

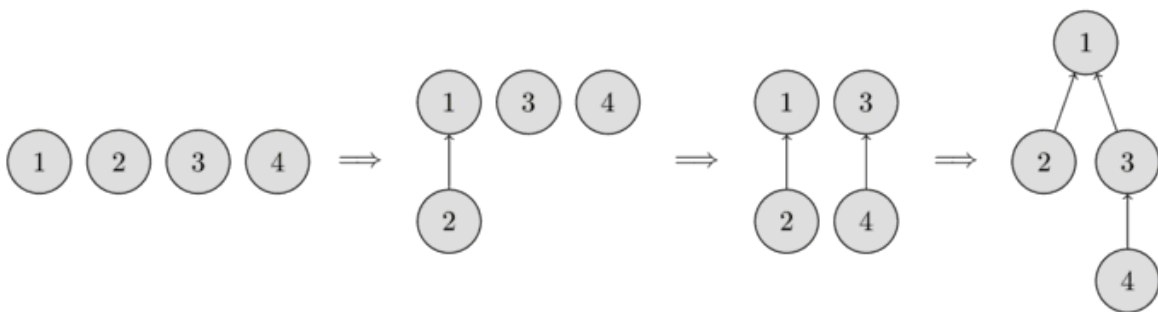
b. **find_set(v)** (Encontrar Conjunto)

Devuelve el representante (o la "raíz") del conjunto al que pertenece el elemento v . El representante es una clave o un identificador único que define todo el conjunto.

- **Uso:** Dos elementos a y b están en el mismo conjunto si y solo si $\text{find_set}(a) = \text{find_set}(b)$.

c. **union_sets(a, b)** (Unir Conjuntos)

Reemplaza el conjunto que contiene a a y el conjunto que contiene a b por su unión. Si a y b ya están en el mismo conjunto (es decir, tienen el mismo representante), no se hace nada.



2. Implementación Naive (Ingenua)

La forma más común de implementar la DSU es usando un array, `parent`, donde `parent[i]` almacena el padre del elemento i . El representante (raíz) de un conjunto es un elemento que

es su propio padre.

```
void make_set(int v) {
    parent[v] = v; // Es su propia raíz
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return find_set(parent[v]); // Recorre hacia la raíz
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b)
        parent[b] = a; // Simplemente conecta una raíz a la otra
}
```

Esta implementación ingenua puede ser muy lenta en el peor de los casos, llevando a árboles muy sesgados y operaciones $O(N)$. Por ello, se utilizan optimizaciones.

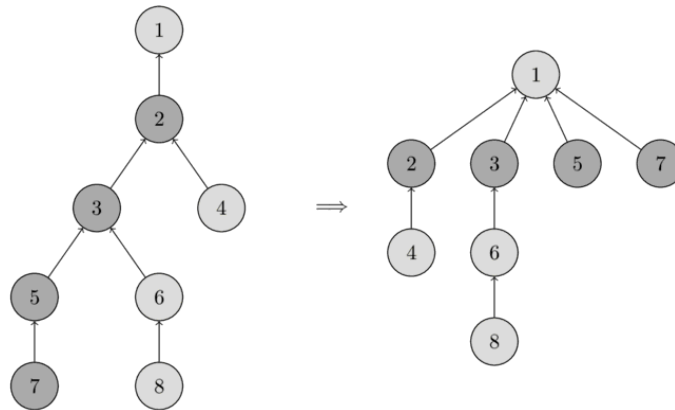
3. Optimizaciones de Rendimiento

Al aplicar dos heurísticas simples, la complejidad de tiempo amortizado para las operaciones `find_set` y `union_sets` se reduce a ser casi constante, $O(\alpha(N))$, donde α es la función inversa de Ackermann, que crece extremadamente lento.

a. Compresión de Trayectoria (Path Compression)

Se utiliza en la operación `find_set`. Después de encontrar la raíz, todos los nodos a lo largo de la ruta se reasignan para que apunten directamente a la raíz. Esto "aplana" el árbol, asegurando que las búsquedas futuras sean más rápidas.

```
int find_set(int v) {
    if (v == parent[v])
        return v;
    // Asigna el padre de 'v' a la raíz real
    return parent[v] = find_set(parent[v]);
}
```



b. Unión por Tamaño o Rango (Union by Size/Rank)

Se utiliza en la operación `union_sets` para evitar que el árbol se vuelva demasiado alto.

- **Unión por Tamaño:** Siempre se adjunta el árbol más pequeño al árbol más grande. El tamaño total del árbol más grande se actualiza.
- **Unión por Rango:** Se utiliza una heurística basada en la "profundidad" (o rango/altura) del árbol, adjuntando el árbol de menor rango al de mayor rango.

4. Implementación Final (Con Optimizaciones)

A continuación, se muestra una implementación eficiente usando la **Unión por Tamaño** y la **Compresión de Trayectoria**.

```
// Se requieren dos arrays: parent[] y size[]
// size[v] almacena el número de elementos en el conjunto representado por v.
std::vector<int> parent;
std::vector<int> size;

void make_set(int v) {
    parent[v] = v;
    size[v] = 1;
}

int find_set(int v) {
    // 1. Compresión de Trayectoria
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}
```

```

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);

    if (a != b) {
        // 2. Unión por Tamaño: Adjuntar el árbol más pequeño al más grande
        if (size[a] < size[b])
            std::swap(a, b); // Asegura que 'a' es el representante del
conjunto más grande

        parent[b] = a;      // El conjunto más pequeño (b) se une a 'a'
        size[a] += size[b]; // Se actualiza el tamaño del nuevo conjunto (a)
    }
}

```